ASSIGNMENT – 3

DSA0101-OBJECT ORIENTED PROGRAMMING USING C++

Student Details	Information
Name	Shahana K
Register Number	192524032
Course	B.Tech – Artificial Intelligence and Data Science
Subject	Object Oriented Programming Using C++
Assignment Number	3

Submission note: The programs are independent C++17 programs. Compile and run each question separately. Experimental timings and sanitizer observations are explained with reproducible commands; machine-dependent measurements should be filled using the actual execution environment.

Contents

- Q1 – Constructors and Constructor Overloading
- Q2 – Copy Constructors: Shallow and Deep Copies
- Q3 – Dynamic Constructors and Destructors
- Q4 – Arithmetic and Stream Operator Overloading
- Q5 – Increment, Comparison and Subscript Operators
- Q6 – Rules of Operator Overloading
- Q7 – Complex-Number Class from First Principles

Q1 – Constructors and Constructor Overloading (6 Marks)

Problem context: A library creates complete, donated and initially empty Book records. Invalid copy counts must never enter a valid Book object.

1. Complete C++ Program

```
#include <iostream>
#include <string>
#include <stdexcept>
using namespace std;
```

```
class Book {
    string title;
    string author;
    int copies;

    static int validCopies(int n) {
        if (n < 0) throw invalid_argument("Copies cannot be negative");
        return n;
    }

public:
    // Empty record
    Book() : Book("Untitled", "Unknown", 0) {}

    // Donated book: title only
    explicit Book(const string& t) : Book(t, "Unknown", 1) {}

    // Full details
    Book(const string& t, const string& a, int n)
        : title(t), author(a), copies(validCopies(n)) {}

    void setCopies(int n) { copies = validCopies(n); }

    void print() const {
        cout << "Title: " << title << ", Author: " << author
             << ", Copies: " << copies << '\n';
    }
};

int main() {
    try {
        Book empty;
```

```
        Book donated("The Alchemist");
        Book complete("Wings of Fire", "A.P.J. Abdul Kalam", 5);

        empty.print();
        donated.print();
        complete.print();

        Book invalid("Bad Record", "Unknown", -2);
        invalid.print();
    } catch (const invalid_argument& e) {
        cout << "Error: " << e.what() << '\n';
    }

    // Arrays require a default constructor for value-initialized elements.
    Book shelf[3];
    return 0;
}
```

2. Expected output

```
Title: Untitled, Author: Unknown, Copies: 0
Title: The Alchemist, Author: Unknown, Copies: 1
Title: Wings of Fire, Author: A.P.J. Abdul Kalam, Copies: 5
Error: Copies cannot be negative
```

3. Design and analysis

- The compiler supplies an implicit default constructor only when the class declares no constructor at all. Once any constructor is declared, the compiler does not automatically add a no-argument constructor.
- Member initializer lists construct members directly. They are required for const members and references, avoid default-construct-then-assign work, and are generally clearer and more efficient.
- Members are initialised in the order in which they are declared in the class, not the order written in the initializer list. Write the list in declaration order to avoid confusion and -Wreorder warnings.

- Constructor delegation uses Book(...): Book(...) so validation and common initialization are implemented in one place.
- The title-only constructor is explicit to prevent an accidental string-to-Book conversion.

Object	Constructor path	Final state
empty	Book() delegates to full constructor	Untitled / Unknown / 0
donated	Book(title) delegates to full constructor	The Alchemist / Unknown / 1
complete	Book(title, author, copies)	Wings of Fire / A.P.J. Abdul Kalam / 5

4. Compiler constructor and array investigation

With Book shelf[3]; the default constructor is called three times. If Book() is removed while the other constructors remain, this array declaration fails because the compiler has no zero-argument constructor to call. A diagnostic similar to “no matching function for call to Book::Book()” is expected.

To demonstrate initialization-order warnings, declare int copies before string title, then deliberately write an initializer list as title(t), copies(n). Compile with g++ -std=c++17 -Wall -Wextra -Wreorder file.cpp. The compiler warns that copies is initialized before title despite the written list order.

5. explicit demonstration

```
class Book {
public:
    Book(const std::string& title) {}
};

void receive(Book b) {}
int main() {
    receive("Dune"); // Compiles when the constructor is not explicit.
}
```

If Book(const std::string&) is marked explicit, receive("Dune") fails because implicit conversion is prohibited. The explicit form receive(Book("Dune")) still compiles.

6. Alternative: default arguments and complexity

A single constructor such as Book(string title = "Untitled", string author = "Unknown", int copies = 0) can represent all three cases. Validation must still occur in that constructor. Both designs create an object in O(1) time and O(1) additional space. A single validating constructor checks copies in one place; overloaded constructors with delegation also centralize the check in the target constructor. Overloads communicate intent more clearly, while defaults reduce the number of declarations.

Q2 – Copy Constructors: Shallow and Deep Copies (7 Marks)

Problem context: A Prescription owns a dynamically allocated array of medicine names. Copies must be independent and memory must be released exactly once.

1. Completed TODO program – deep-copy version

```
#include <iostream>
#include <string>
using namespace std;

class Prescription {
    int n;
    string* medicines;

public:
    explicit Prescription(int count)
        : n(count), medicines(count > 0 ? new string[count] : nullptr) {
        if (count < 0) throw invalid_argument("Negative medicine count");
    }

    // TODO 1: deep-copy constructor
    Prescription(const Prescription& other)
        : n(other.n), medicines(n > 0 ? new string[n] : nullptr) {
        for (int i = 0; i < n; ++i) medicines[i] = other.medicines[i];
        cout << "[copy constructor]\n";
    }
```

```
    // TODO 2: copy assignment with self-assignment protection
    Prescription& operator=(const Prescription& other) {
        if (this == &other) return *this;
        string* fresh = other.n > 0 ? new string[other.n] : nullptr;
        for (int i = 0; i < other.n; ++i) fresh[i] = other.medicines[i];
        delete[] medicines;
        medicines = fresh;
        n = other.n;
        return *this;
    }

    // TODO 3: destructor
    ~Prescription() { delete[] medicines; }

    void set(int i, const string& m) { medicines[i] = m; }
    string get(int i) const { return medicines[i]; }
    int size() const { return n; }
};

void showByValue(Prescription p) { cout << "Passed by value\n"; }
void showByReference(const Prescription& p) { cout << "Passed by const
reference\n"; }

int main() {
    Prescription p(2);
    p.set(0, "Paracetamol");
    p.set(1, "ORS");

    Prescription q = p;
    q.set(1, "Cetirizine");

    Prescription r(1);
    r = p;
```

```
    p = p; // self-assignment test

    cout << p.get(0) << ' ' << p.get(1) << " | "
         << q.get(0) << ' ' << q.get(1) << " | "
         << r.get(0) << ' ' << r.get(1) << '\n';

    showByValue(p);
    showByReference(p);
}
```

2. Expected output

```
[copy constructor]
Paracetamol ORS | Paracetamol Cetirizine | Paracetamol ORS
[copy constructor]
Passed by value
Passed by const reference
```

The exact placement of copy messages depends on which calls are present. Passing by value requires a copy (unless an allowed elision applies); passing by const reference does not copy the Prescription object.

3. Shallow-copy versus deep-copy memory diagrams

Shallow copy (incorrect)	**Deep copy (correct)**
p.medicines ──► [Paracetamol, ORS] ◄── q.medicines	p.medicines ──► [Paracetamol, ORS]
Both objects point to the same allocation.	q.medicines ──► [Paracetamol, ORS] (separate allocation).
Changing q changes p; both destructors call delete[] on the same pointer.	Changing q does not affect p; each destructor releases its own array.

4. Rule of Three and copy-constructor calls

The Rule of Three says that if a class needs a user-defined destructor, copy constructor, or copy-assignment operator because it owns a resource, it usually needs all three. The copy constructor is used when an object is initialized from another object, when an object is passed by value, and when an object is returned by value (although copy elision may remove the physical copy).

The shallow version copies only the pointer. At destruction, two objects attempt to delete the same allocation, producing a double-free error; modifications also leak across copies. A deep copy allocates a distinct array and copies each string.

5. Sanitizer and experiments

```
g++ -std=c++17 -g -O1 -fsanitize=address -fno-omit-frame-pointer q2.cpp -o
q2
./q2
```

Correct deep-copy version: AddressSanitizer should report no memory error. Shallow-copy version: typically reports attempting double-free on the medicine array. Exact diagnostic wording and addresses depend on compiler/runtime.

Experiment	**Expected observation**
10 medicines	Copy work proportional to 10 elements
1,000 medicines	More copying than 10; linear growth
100,000 medicines	Largest copy time and memory use
p = p	No change; self-assignment guard returns immediately
Pass by value	One logical copy at the call boundary unless elision applies
Pass by const reference	No Prescription copy

For n medicine strings, deep copy is O(n) time and O(n) extra space. A shallow pointer copy is O(1) but is incorrect for exclusive ownership. With std::vector<string>, compiler-generated copy operations already perform independent element copies (Rule of Zero); no custom destructor/copy operations are needed.

6. When is compiler-generated copying safe?

It is safe when every data member already has correct value semantics (for example int, string, vector, and other well-behaved value types). Write custom copying when the class directly owns a raw resource such as a new[] allocation, file handle, or unique external resource. Prefer Rule of Zero containers/smart pointers whenever possible.

Q3 – Dynamic Constructors and Destructors (7 Marks)

Problem context: An auditorium event owns a dynamically allocated two-dimensional seat map. The map must be released automatically when the Event is destroyed.

1. Complete C++ program

```
#include <iostream>
#include <string>
using namespace std;

class Event {
    string name;
    int rows, seatsPerRow;
    bool** seats;
    int booked;

public:
    Event(const string& eventName, int r, int s)
        : name(eventName), rows(r < 0 ? 0 : r),
          seatsPerRow(s < 0 ? 0 : s), seats(nullptr), booked(0) {
        if (rows > 0 && seatsPerRow > 0) {
            seats = new bool*[rows]{};
            try {
                for (int i = 0; i < rows; ++i)
                    seats[i] = new bool[seatsPerRow]{false};
            } catch (...) {
                for (int i = 0; i < rows; ++i) delete[] seats[i];
                delete[] seats;
                throw;
            }
        }
        cout << "Created: " << name << '\n';
    }
```

```
    bool book(int r, int s) {
        if (!seats || r < 0 || r >= rows || s < 0 || s >= seatsPerRow)
            return false;
        if (seats[r][s]) return false;
        seats[r][s] = true; ++booked; return true;
    }

    bool cancel(int r, int s) {
        if (!seats || r < 0 || r >= rows || s < 0 || s >= seatsPerRow)
            return false;
        if (!seats[r][s]) return false;
        seats[r][s] = false; --booked; return true;
    }

    void summary() const {
        cout << name << ": " << booked << " seat(s) booked\n";
    }

    ~Event() {
        for (int i = 0; i < rows; ++i) delete[] seats[i];
        delete[] seats;
        cout << "Closed: ";
        summary();
    }
};

int main() {
    Event stackEvent("Stack Event", 2, 3);
    stackEvent.book(0, 1);
    stackEvent.book(1, 2);
    stackEvent.cancel(0, 1);

    {
```

```
        Event blockEvent("Block Event", 1, 2);
        blockEvent.book(0, 0);
    }

    Event* heapEvent = new Event("Heap Event", 2, 2);
    heapEvent->book(1, 1);
    delete heapEvent;

    Event zeroEvent("Zero Row Event", 0, 5);
    cout << "Zero-row booking: " << zeroEvent.book(0, 0) << '\n';
    return 0;
}
```

2. Representative output

```
Created: Stack Event
Created: Block Event
Closed: Block Event: 1 seat(s) booked
Created: Heap Event
Closed: Heap Event: 1 seat(s) booked
Created: Zero Row Event
Zero-row booking: 0
Closed: Zero Row Event: 0 seat(s) booked
Closed: Stack Event: 1 seat(s) booked
```

The block object is destroyed when its block ends. The heap object is destroyed at delete. The stackEvent and zeroEvent are destroyed in reverse construction order at the end of main. The output above follows the exact order of these operations.

3. Lifetime, allocation and safety

- Automatic local objects are destroyed in reverse order of construction when their scope ends.
- A block-scope object is destroyed at the closing brace of that block.
- A dynamically allocated object remains alive until delete is executed. Forgetting delete leaks the object and its seat arrays.

- Each new[] must match delete[]. Using scalar delete on an array is undefined behaviour; use delete[] for every array allocation.
- A destructor has no parameters and no return type, and a class has only one destructor. Destruction cannot be selected by overload arguments.

Memory layout: Event object stores rows, seatsPerRow, booked and a bool** pointer. That pointer refers to an array of row pointers; each row pointer refers to a separate bool[seatsPerRow] allocation.

4. Memory and alternative layouts

Map size	**Seat cells**	**Approx. bool payload (1 byte per bool)**
10 × 20	200	200 bytes
100 × 200	20,000	20,000 bytes
1,000 × 2,000	2,000,000	2,000,000 bytes (~1.91 MiB)

The pointer-per-row layout makes rows + 1 allocations and adds rows × sizeof(bool*) bytes for row pointers. A contiguous block uses one allocation and index r * seatsPerRow + s; access remains O(1) in both. vector<vector<bool>> or vector<vector<char>> manages memory automatically and is safer. RAII ties resource release to object lifetime, preventing forgotten cleanup on normal scope exit.

5. Sanitizer command

```
g++ -std=c++17 -g -O1 -fsanitize=address -fno-omit-frame-pointer q3.cpp -o
q3
./q3
```

With the destructor omitted, the sanitizer should report leaked allocations (LeakSanitizer support depends on platform). With the destructor shown, the program should finish without leaks or invalid frees. Zero rows are handled without allocating a row-pointer array.

Q4 – Overloading Arithmetic and Stream Operators (7 Marks)

Problem context: A billing counter represents money as an integer number of paise to avoid floating-point rounding errors.

1. Completed TODO program and extended tests

```
#include <iomanip>
#include <iostream>
#include <stdexcept>
using namespace std;

class Money {
    long long paise;

public:
    Money(long long rupees = 0, int p = 0) : paise(rupees * 100 + p) {}

    Money operator+(const Money& other) const {
        return Money(0, static_cast<int>(paise + other.paise));
    }

    Money operator-(const Money& other) const {
        Money result;
        result.paise = paise - other.paise;
        return result;
    }

    Money& operator+=(const Money& other) {
        paise += other.paise;
        return *this;
    }

    bool operator==(const Money& other) const {
        return paise == other.paise;
```

```
    }

    bool operator<(const Money& other) const {
        return paise < other.paise;
    }

    friend ostream& operator<<(ostream& out, const Money& m) {
        long long magnitude = m.paise < 0 ? -m.paise : m.paise;
        if (m.paise < 0) out << "-";
        out << "Rs " << magnitude / 100 << '.'
            << setw(2) << setfill('0') << magnitude % 100
            << setfill(' ');
        return out;
    }

    friend istream& operator>>(istream& in, Money& m) {
        long long rupees; int p;
        if (in >> rupees >> p) m.paise = rupees * 100 + p;
        return in;
    }
};

int main() {
    Money a(120, 75), b(79, 25);
    Money c = a + b;
    cout << c << '\n' << boolalpha << (c == Money(200)) << '\n';

    Money carry = Money(0, 75) + Money(0, 50);
    cout << carry << '\n';

    Money negative = Money(5, 0) - Money(8, 25);
    cout << negative << '\n';
```

```
    Money bill(100, 0);
    bill += a;
    cout << "Bill: " << bill << '\n';
}
```

2. Expected output

```
Rs 200.00
true
Rs 1.25
-Rs 3.25
Bill: Rs 220.75
```

The constructor in the supplied template stores rupees × 100 + paise. The addition result is represented internally in paise; a production-quality constructor should normalize arbitrary paise and guard integer overflow. The sample keeps the interface compact for the assignment.

3. Operator design and translation

- Binary + returns a new Money value and does not modify either operand. += modifies the left operand and returns Money& (*this), allowing chaining without an unnecessary copy.
- Arithmetic and comparison operators can be members because the left operand is a Money object. Stream insertion and extraction are non-members (commonly friends) because the left operand is ostream/istream, not Money.
- Friend access lets << and >> access the private paise field while retaining the natural syntax cout << amount and cin >> amount.
- The expression a + b is translated conceptually to a.operator+(b) when + is a member.

4. Engineering decision and complexity

Multiplying one Money amount by another Money amount is generally dimensionally meaningless (currency squared). A scalar multiplication operator such as Money × quantity may be useful, with explicit rounding rules. Every basic arithmetic/comparison operation is O(1) time and O(1) extra space.

Using friend non-member operators also has O(1) time and space. Non-member operators allow symmetric expressions such as Money(100) + m or 100 + m when a suitable non-explicit conversion is provided. Named functions add() and equals() are equally efficient but less idiomatic. A double-based

implementation can accumulate rounding error; integer paise is exact for whole-paise values. Benchmark one million additions using std::chrono, but report the measured value from the target machine rather than assuming a universal time.

5. Compiler check

```
g++ -std=c++17 -Wall -Wextra q4.cpp -o q4
./q4
```

Q5 – Overloading Increment, Comparison and Subscript Operators (6 Marks)

Problem context: A music player needs a Time value that advances by one second and a Playlist with checked indexed access.

1. Complete C++ program

```
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
using namespace std;

class Time {
    int seconds; // seconds since 00:00:00

public:
    explicit Time(int s = 0) : seconds((s % 86400 + 86400) % 86400) {}

    Time& operator++() { // prefix
        seconds = (seconds + 1) % 86400;
        return *this;
    }

    Time operator++(int) { // postfix
        Time old = *this;
        ++(*this);
        return old;
    }

    bool operator==(const Time& other) const {
        return seconds == other.seconds;
    }

    bool operator<(const Time& other) const {
```

```
        return seconds < other.seconds;
    }

    friend ostream& operator<<(ostream& out, const Time& t) {
        int h=t.seconds/3600, m=(t.seconds/60)%60, s=t.seconds%60;
        return out << (h<10?"0":"") << h << ':'
                   << (m<10?"0":"") << m << ':'
                   << (s<10?"0":"") << s;
    }
};

class Playlist {
    vector<string> tracks;

public:
    Playlist(initializer_list<string> t) : tracks(t) {}

    string& operator[](int i) {
        if (i < 0 || i >= static_cast<int>(tracks.size()))
            throw out_of_range("Playlist index out of range");
        return tracks[i];
    }

    const string& operator[](int i) const {
        if (i < 0 || i >= static_cast<int>(tracks.size()))
            throw out_of_range("Playlist index out of range");
        return tracks[i];
    }

    int size() const { return static_cast<int>(tracks.size()); }
};

int main() {
```

```
    Time t(3599);
    Time a = t++;
    Time b = ++t;
    cout << "a=" << a << ", b=" << b << ", t=" << t << '\n';

    Time last(59*60+59);
    ++last;
    cout << "Rollover: " << last << '\n';

    Playlist playlist{"Song A", "Song B", "Song C"};
    for (int i=0; i<playlist.size(); ++i)
        cout << i << ": " << playlist[i] << '\n';

    try { cout << playlist[-1] << '\n'; }
    catch (const out_of_range& e) { cout << e.what() << '\n'; }

    try { cout << playlist[playlist.size()] << '\n'; }
    catch (const out_of_range& e) { cout << e.what() << '\n'; }
}
```

2. Expected output

```
a=00:59:59, b=01:00:01, t=01:00:01
Rollover: 01:00:00
0: Song A
1: Song B
2: Song C
Playlist index out of range
Playlist index out of range
```

3. Predict-the-output supplied Counter program

```
#include <iostream>
struct Counter {
```

```
    int v;
    Counter& operator++() { ++v; return *this; }
    Counter operator++(int) { Counter old = *this; ++v; return old; }
};
int main() {
    Counter c{5};
    Counter a = c++;
    Counter b = ++c;
    std::cout << a.v << ' ' << b.v << ' ' << c.v << '\n';
}
```

Predicted and actual output: 5 7 7. First c++ returns a copy containing 5, then c becomes 6. Next ++c increments c to 7 and returns a reference to the updated object, so b receives 7.

4. Design and complexity

- The compiler distinguishes prefix ++x from postfix x++ using the dummy int parameter in operator++(int).
- Postfix must return the old value, so it saves and returns a copy. Prefix returns *this by reference and avoids that copy; this is why ++x is often preferred for class types.
- Playlist has both non-const and const operator[] overloads: the non-const version returns string&, allowing modification; the const version returns const string&, preventing modification through a const playlist.
- Out-of-range access should fail safely. This solution throws std::out_of_range; printing an error and continuing is another possible policy, but silently accessing invalid memory is unsafe.

Increment, comparison and indexed access are O(1). Checked access adds one bounds comparison, still O(1). Benchmark 10,000,000 prefix/postfix operations at -O0 and -O2 with std::chrono; ensure the result is observable so the compiler cannot remove the loop. Exact times depend on compiler, CPU and optimization settings.

```
g++ -std=c++17 -O0 q5.cpp -o q5_O0
g++ -std=c++17 -O2 q5.cpp -o q5_O2
```

Q6 – The Rules of Operator Overloading (7 Marks)

Problem context: An SMS gateway Text class owns a dynamic character array and supports copying, concatenation, comparison and printing.

1. Complete C++ program

```
#include <cstring>
#include <iostream>
#include <string>
using namespace std;

class Text {
    char* data;

public:
    Text(const char* s = "") {
        if (!s) s = "";
        data = new char[strlen(s) + 1];
        strcpy(data, s);
    }

    Text(const Text& other) {
        data = new char[strlen(other.data) + 1];
        strcpy(data, other.data);
    }

    Text& operator=(const Text& other) {
        if (this == &other) return *this;
        char* fresh = new char[strlen(other.data) + 1];
        strcpy(fresh, other.data);
        delete[] data;
        data = fresh;
        return *this;
    }
```

```
    ~Text() { delete[] data; }

    Text operator+(const Text& other) const {
        size_t n = strlen(data) + strlen(other.data);
        char* buffer = new char[n + 1];
        strcpy(buffer, data);
        strcat(buffer, other.data);
        Text result(buffer);
        delete[] buffer;
        return result;
    }

    bool operator==(const Text& other) const {
        return strcmp(data, other.data) == 0;
    }

    bool operator!=(const Text& other) const {
        return !(*this == other);
    }

    friend ostream& operator<<(ostream& out, const Text& t) {
        return out << t.data;
    }
};

int main() {
    Text a("Hello "), b("World"), c("!");
    Text result = a + b + c;
    cout << result << '\n';

    Text empty("");
    Text longMessage("This is a longer SMS message used for testing.");
```

```
    cout << "Empty=[" << empty << "]\n";
    cout << longMessage << '\n';

    a = a; // self-assignment
    Text copy = a;
    cout << boolalpha << (copy == a) << ' ' << (copy != b) << '\n';
}
```

2. Expected output

```
Hello World!
Empty=[]
This is a longer SMS message used for testing.
true true
```

3. Operator rules

- Operators that cannot be overloaded include . (member access), .* (pointer-to-member access), :: (scope resolution), ?: (conditional), sizeof, and typeid. Their syntax and compile-time semantics are fundamental language operations.
- Overloading cannot change an operator's precedence, associativity, or arity. At least one operand must have a class or enumeration type; operators cannot be redefined for two built-in operands.
- operator= must be a non-static member function because the language requires assignment to be implemented as a member of the left-hand class.
- A friend is useful when a non-member operator needs private access, especially stream insertion where the left operand is ostream. Friends should be used only when needed.

4. Copy count and complexity

In a + b + c, the expression groups as (a + b) + c. Each + creates a result containing the concatenated characters. A copy constructor may be invoked when a temporary is materialized or copied, but modern C++ compilers can elide some copies. Instrument the copy constructor and run with compiler flags both with and without copy elision if investigating counts.

Concatenating strings of lengths n1, n2 and n3 with repeated + copies approximately (n1+n2) characters into the first result and then (n1+n2+n3) into the second, for total work O(n1+n2+n3) and temporary

storage O(n1+n2+n3). An implementation of operator+ in terms of += on a copy has similar asymptotic complexity and can reduce duplicated logic. std::string is normally preferred because it already manages capacity, allocation and exception safety.

5. Rule of Three and sanitizer

Text defines destructor, copy constructor and copy assignment because it owns a raw char array. The self-assignment guard prevents deleting the source data before copying it. Run:

```
g++ -std=c++17 -g -O1 -fsanitize=address -fno-omit-frame-pointer q6.cpp -o
q6
./q6
```

The completed program should have no invalid read, double-free or leak. For 100,000 concatenations, compare this implementation with std::string using the same input sizes and std::chrono. Do not treat a single machine's timing as a universal result.

6. What this class teaches

Writing Text demonstrates ownership, deep copying, self-assignment protection, operator design and the cost of temporary objects. A custom class is justified when specialized storage, encoding, memory limits, instrumentation or interoperability requirements demand it. For ordinary text processing, std::string is safer and more maintainable (Rule of Zero).

Q7 – Design a Complex-Number Class from First Principles (10 Marks)

Problem context: An AC impedance calculator represents resistance, inductive reactance and capacitive reactance as complex numbers.

1. Requirements and design

- Represent a complex impedance with real and imaginary double components.
- Provide default, real-only, and real-plus-imaginary constructors.
- Support +, -, *, /, == and stream output.
- Calculate magnitude as sqrt(real² + imaginary²).
- Detect division by a zero complex denominator and report it safely.

Operator	**Formula**
a + b	(a.real+b.real, a.imag+b.imag)
a - b	(a.real-b.real, a.imag-b.imag)
a * b	(ac-bd, ad+bc)
a / b	((ac+bd)/(c²+d²), (bc-ad)/(c²+d²))
Magnitude	sqrt(real² + imag²)

2. Complete C++ program

```
#include <cmath>
#include <iostream>
#include <stdexcept>
#include <iomanip>
using namespace std;

class Complex {
    double re, im;

public:
    Complex() : re(0.0), im(0.0) {}
    Complex(double real) : re(real), im(0.0) {}
    Complex(double real, double imag) : re(real), im(imag) {}
```

```
double real() const { return re; }
double imag() const { return im; }

Complex operator+(const Complex& z) const {
    return Complex(re + z.re, im + z.im);
}

Complex operator-(const Complex& z) const {
    return Complex(re - z.re, im - z.im);
}

Complex operator*(const Complex& z) const {
    return Complex(re*z.re - im*z.im, re*z.im + im*z.re);
}

Complex operator/(const Complex& z) const {
    double denominator = z.re*z.re + z.im*z.im;
    if (denominator == 0.0)
        throw domain_error("Division by zero complex impedance");
    return Complex((re*z.re + im*z.im)/denominator,
                   (im*z.re - re*z.im)/denominator);
}

bool operator==(const Complex& z) const {
    const double eps = 1e-9;
    return fabs(re-z.re) < eps && fabs(im-z.im) < eps;
}

double magnitude() const { return sqrt(re*re + im*im); }

friend ostream& operator<<(ostream& out, const Complex& z) {
    out << fixed << setprecision(3) << z.re;
```

```
        if (z.im >= 0) out << " + " << z.im << "j";
        else out << " - " << -z.im << "j";
        return out;
    }
};

int main() {
    const double w = 100.0;
    const double L = 0.01;
    const double C = 0.001;

    Complex R(10.0, 0.0);
    Complex ZL(0.0, w*L);
    Complex ZC(0.0, -1.0/(w*C));

    Complex series = R + ZL + ZC;
    cout << "R = " << R << '\n';
    cout << "ZL = " << ZL << '\n';
    cout << "ZC = " << ZC << '\n';
    cout << "Series impedance = " << series << '\n';
    cout << "Series magnitude = " << series.magnitude() << '\n';

    Complex parallel = (R * ZL) / (R + ZL);
    cout << "R parallel L = " << parallel << '\n';

    try {
        Complex invalid = R / Complex(0.0, 0.0);
        cout << invalid << '\n';
    } catch (const domain_error& e) {
        cout << "Error: " << e.what() << '\n';
    }
}
```

3. Verified values and hand calculation

For the chosen values ω = 100 rad/s, R = 10 Ω, L = 0.01 H and C = 0.001 F: ZR = 10 + 0j; ZL = jωL = 0 + 1j; ZC = −j/(ωC) = 0 − 10j. Series impedance = 10 − 9j Ω and magnitude = $\sqrt{(10^2 + 9^2)} = \sqrt{181} \approx 13.454$ Ω.

For R in parallel with L: (10 × j1)/(10 + j1) = (j10)/(10+j1). Multiplying numerator and denominator by (10−j1) gives (10 + j100)/101 ≈ 0.099 + 0.990j Ω. Division by Complex(0,0) throws domain_error and is caught in main.

4. Sample output

```
R = 10.000 + 0.000j
ZL = 0.000 + 1.000j
ZC = 0.000 - 10.000j
Series impedance = 10.000 - 9.000j
Series magnitude = 13.454
R parallel L = 0.099 + 0.990j
Error: Division by zero complex impedance
```

5. Design defence and comparison

- The default constructor creates 0 + 0j; the one-argument constructor represents a real-only impedance; the two-argument constructor supports arbitrary rectangular form.
- Arithmetic operators are members because the left operand is Complex. Stream insertion is a friend non-member because the left operand is ostream.
- std::complex<double> already implements standard complex arithmetic and is preferred in production. Both representations use two doubles and arithmetic operations take O(1) time and O(1) space.
- The custom class demonstrates formulas and allows tailored output and error policy. std::complex division follows the standard library's floating-point semantics; explicit application-level validation may still be required for a zero impedance.
- One limitation is exact floating-point equality: a tolerance-based comparison is more suitable for computed values. The fixed absolute tolerance shown is simple but may need scaling for very large or very small impedances.

6. Compile command

```
g++ -std=c++17 -Wall -Wextra q7.cpp -o q7
./q7
```

Overall Compilation and Verification Checklist

Question	Compile/run command	Key verification
Q1	g++ -std=c++17 -Wall -Wextra q1.cpp -o q1 && ./q1	All constructor forms; negative copies rejected; array default construction.
Q2	g++ -std=c++17 -g -fsanitize=address q2.cpp -o q2 && ./q2	Independent copies; self-assignment; no double free.
Q3	g++ -std=c++17 -g -fsanitize=address q3.cpp -o q3 && ./q3	Stack/block/heap lifetime; zero rows; no leaks.
Q4	g++ -std=c++17 -Wall -Wextra q4.cpp -o q4 && ./q4	Paise carry, negative amount, stream output.
Q5	g++ -std=c++17 -Wall -Wextra q5.cpp -o q5 && ./q5	Prefix/postfix values, rollover, invalid indices.
Q6	g++ -std=c++17 -g -fsanitize=address q6.cpp -o q6 && ./q6	Deep copy, self-assignment, long/empty strings.
Q7	g++ -std=c++17 -Wall -Wextra q7.cpp -o q7 && ./q7	Series/parallel values and division-by-zero handling.

Experimental results: how to report honestly

Sanitizer results, execution times and memory measurements depend on the operating system, compiler version, CPU and runtime. The expected outcomes in this report describe what a correct implementation should demonstrate; they are not presented as measured timings from your computer. Run the listed commands and paste your actual terminal output/screenshots into the evidence section before submission.

Suggested GitHub repository structure

```
Assignment-3-OOP-CPP/
|-- README.md
```

```
|-- Q1_Constructors/
|   `-- q1.cpp
|-- Q2_Copy_Constructors/
|   `-- q2.cpp
|-- Q3_Dynamic_Memory/
|   `-- q3.cpp
|-- Q4_Money_Operators/
|   `-- q4.cpp
|-- Q5_Increment_Playlist/
|   `-- q5.cpp
|-- Q6_Text_Class/
|   `-- q6.cpp
|-- Q7_Complex_Class/
|   `-- q7.cpp
`-- Evidence/
    |-- outputs/
    |-- sanitizer/
    `-- timings/
```

Final conclusion

The seven tasks demonstrate the central C++ object-oriented concepts: constructor design, safe ownership and copying, RAII, operator overloading, const correctness, and class design from mathematical requirements. The safest modern C++ approach is to prefer standard library value types and RAII containers (Rule of Zero), and to implement custom resource management only when the design genuinely requires it.